

Name: Duan Nguyen

```
In [1]: import numpy as np
```

Exercise 1:

```
In [12]: A = [1.0, 1.0]
B = [1.5, 0.5]
C = [2.0, 1.0]
D = [2.5, 2.0]
Z = [0.0, 0.0]

A_prime = [-0.9, 0.8]
B_prime = [-0.1, 1.3]
C_prime = [-0.4, 1.9]
D_prime = [-1.25, 2.55]
print(A+Z)
```

```
[1.0, 1.0, 0.0, 0.0]
```

Step 1

$$\begin{bmatrix} m_{11} & m_{12} & m_{21} & m_{22} \end{bmatrix} = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

$$\begin{bmatrix} m_{11} & m_{12} & m_{21} & m_{22} \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix}$$

Step 2

$$\begin{bmatrix} Q \\ b \end{bmatrix} = \begin{bmatrix} 1.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 1.0 \\ 1.5 & 0.5 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.5 & 0.5 \\ 2.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 2.0 & 1.0 \\ 2.5 & 2.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 2.5 & 2.0 \end{bmatrix} \begin{bmatrix} -0.9 \\ 0.8 \\ -0.1 \\ 1.3 \\ -0.4 \\ 1.9 \\ -1.25 \\ 2.55 \end{bmatrix}$$

```
In [3]: Q = [A + Z,
             Z + A,
             B + Z,
             Z + B,
             C + Z,
             Z + C,
             D + Z,
             Z + D]
Q = np.array(Q)

b = [A_prime + B_prime + C_prime + D_prime]
b = np.array(b).T
```

Step 3

$$M = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} = \begin{bmatrix} 0.332 & -1.081 \\ 0.876 & 0.126 \end{bmatrix}$$

```
In [14]: M = np.linalg.lstsq(Q, b, rcond=-1)[0]
M = M.reshape((2,2))
print(M)
```

```
[[ 0.332 -1.0808]
 [ 0.876  0.1256]]
```

Exercise 2

```
In [6]: import cv2 as cv
```

```

def get_keypoint(left_img, right_img):
    l_img = cv.cvtColor(left_img, cv.COLOR_BGR2GRAY)
    r_img = cv.cvtColor(right_img, cv.COLOR_BGR2GRAY)

    sift = cv.SIFT_create()
    key_points1, descriptor1 = sift.detectAndCompute(l_img, None)
    key_points2, descriptor2 = sift.detectAndCompute(r_img, None)

    return key_points1, descriptor1, key_points2, descriptor2

def match_keypoints(descriptor1, descriptor2):
    FLANN_INDEX_KDTREE = 1
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
    search_params = dict(checks=50)

    flann = cv.FlannBasedMatcher(index_params, search_params)
    matches = flann.knnMatch(descriptor1, descriptor2, k=2)

    good_matches = []
    for m, n in matches:
        if m.distance < 0.6 * n.distance:
            good_matches.append(m)

    return good_matches

img1 = cv.imread('./hw09_image/left.png')
img2 = cv.imread('./hw09_image/right.png')

f, cx, cy = 48.5, 48.5, 48.5
K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])

key_points1, descriptor1, key_points2, descriptor2 = get_keypoint(img1, img2)

good_matches = match_keypoints(descriptor1, descriptor2)

pts1 = np.float32([key_points1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
pts2 = np.float32([key_points2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

F, inlier_mask = cv.findFundamentalMat(pts1, pts2, cv.FM_RANSAC, 2, 0.99, maxIters=1000)

E = K.T @ F @ K
positive_num, R, t, positive_mask = cv.recoverPose(E, pts1, pts2, K, mask=inlier_mask)

P0 = K @ np.eye(3, 4, dtype=np.float32)
Rt = np.hstack((R, t))
P1 = K @ Rt
pts1_inlier = pts1[inlier_mask.ravel() == 1]
pts2_inlier = pts2[inlier_mask.ravel() == 1]
X = cv.triangulatePoints(P0, P1, pts1_inlier, pts2_inlier)
X /= X[3]
X = X.T

```

In [11]: print(X[:5])

```

[[ 47.038296  253.6979   21.815971   1.         ]
 [ 17.213387   60.94946   7.3244987   1.         ]
 [ 10.873277   22.608986   3.0421755   1.         ]
 [  9.893033   20.596962   2.1171954   1.         ]
 [  9.893033   20.596962   2.1171954   1.         ]]

```

What is matrix K in line 35?

K contains the intrinsic matrix that convert 3-D world point into 2-D image pixel. The matrix contains a focal length, f , of 48.5, and camera parameters of (48.5, 48.5).

What type of image keypoint/descriptor is extracted in line 37?

The extracted image keypoints/descriptors are SIFT for both the right and left images to detect their keypoints along with their changes for matching purposes.

What is the Lowe's threshold used for matching keypoints in line 39?

The matching Lowe's threshold is 0.6 for ratio distance between the largest matching distance to its second largest matching distance to filter incorrect matches.

What are variables F and inlier_mask in line 44?

F contains the fundamental matrix of pixels. `inlier_mask` is taking out the outlier through RANSAC

What is E in line 46?

E contains the essential matrix that relates corresponding parts of image points across the two images to estimate relative position/orientation.

What is the objective of line 47?

It computes the relative camera pose/location.

What is the objective of lines 49-56, and what is output X ?

This calculate and find X , which is the 3D triangulated points in the world, using matched features from recovered camera pose.